

Projekt 0

Algorytmy Optymalizacji Inspirowane Naturą

Stanisław Kurzyp - 264477@student.pwr.edu.pl

Prowadzący

– dr inż. Paweł Myszkowski

Spis treści

1	Wprowadzenie	2
1.1	Założenia projektu	2
2	Analiza problemu optymalizacyjnego	2
2.1	Metaheurystyki i algorytmy optymalizacji	2
2.2	cVRP	2
3	Zastosowane algorytmy	3
3.1	Algorytm losowy	3
3.2	Algorytm zachłanny	3
3.3	Algorytm Symulowanego Wyżarzania	3
3.4	Metaheurystyka genetyczna	3
4	Implementacja	4
4.1	Język Go	4
4.2	Implementacje metaheurystyk	5
4.2.1	Realizacja generowania rozwiązań	5
4.2.2	Metaheurystyka Genetyczna	5
4.2.3	Metaheurystyka symulowanego wyżarzania	5
5	Doświadczenie	6
5.1	Dostrajanie parametrów metaheurystyk	6
5.2	Wyniki eksperymentów	7
5.2.1	Eksperyment generalny	7
5.3	Średnie wyniki dla poszczególnych instancji	7
6	Podsumowanie i wnioski	9
A	Suplement: Analiza parametrów algorytmu ewolucyjnego	11
A.1	Wpływ rozmiaru turnieju na działanie EA	11
A.2	Różnica między EA z Tour=1 a Tour _z 1	11
A.3	Poprawność działania ruletki dla problemu cVRP	11
A.4	Czy mutacji może być za mało lub za dużo?	11
A.5	Rola operatora mutacji	11
A.6	Czy krzyżowania może być za mało lub za dużo?	11
A.7	Rola operatora krzyżowania	11
A.8	Konsekwencje wyłączenia krzyżowania i/lub mutacji	11
A.9	Potrzeba selekcji elitarniej	12
A.10	Wpływ rozmiaru populacji na efektywność EA	12

1 Wprowadzenie

Sprawozdanie to dokumentuje projekt wykonany w ramach pierwszego etapu kursu Algorytmy Optymalizacji Inspirowane Naturą w semestrze zimowym 2025/2026.

1.1 Założenia projektu

Celem projektu było zaimplementowanie metaheurystyki genetycznej, która rozwiązuje problem cVRP (*ang. Capacitated Vehicle Routing Problem*). Ponadto w ramach zadania należało również zaimplementować inną metaheurystykę oraz algorytmy **zachłanny** i **losowy**. Na podstawie zgromadzonych wyników w ramach przeprowadzonych badań na zestawie instancji wyciągnięto wnioski i przygotowano ich wizualizację na wykresach.

- Zbadanie działania EA na 7 plikach – instancje podane wraz z treścią. Testy muszą być przeprowadzone na: 2 łatwych, 2 średnich i 2 trudnych instancjach (1 pkt).
- Implementacja i zbadanie algorytmu losowego (1pkt),
- Implementacja i zbadanie algorytmu zachłannego (1pkt),
- Implementacja i zbadanie innej metaheurystyki (2pkt)
- Zbadanie wpływu konfiguracji obu metaheurystyk na ich skuteczność/efektywność (3pkt)
- Porównanie skuteczności wszystkich metod (2pkt)

2 Analiza problemu optymalizacyjnego

Poniższy rozdział w skrócie opisuje problem optymalizacyjny, który jest tematem tego doświadczenia. Ponadto przytacza on najważniejsze pojęcia dotyczące metaheurystyk i algorytmów optymalizacji inspirowanych naturą.

2.1 Metaheurystyki i algorytmy optymalizacji

Metaheurystyki to ogólne strategie rozwiązywania problemów optymalizacyjnych, które nie gwarantują znalezienia rozwiązania optymalnego, ale w rozsądnym czasie są w stanie wygenerować rozwiązania o dobrej jakości. W przeciwieństwie do algorytmów dokładnych, które dla złożonych problemów kombinatorycznych wymagają niepraktycznie długiego czasu obliczeń, metaheurystyki stosują heurystyki wyższego poziomu do eksploracji przestrzeni rozwiązań.

Algorytmy optymalizacji inspirowane naturą stanowią podklasę metaheurystyk, które czerpią inspirację z procesów obserwowanych w przyrodzie. Do tej grupy należą między innymi algorytmy genetyczne (naśladujące ewolucję biologiczną), symulowane wyżarzanie (oparte na procesie krystalizacji metali) czy algorytmy mrówiskowe. Kluczową cechą tych metod jest zdolność do unikania utknięcia w lokalnych optima poprzez kontrolowane wprowadzanie elementu losowości do procesu przeszukiwania.

2.2 cVRP

Problem trasowania pojazdów z ograniczeniami pojemności (*ang. Capacitated Vehicle Routing Problem*, cVRP) jest jednym z najbardziej fundamentalnych wariantów problemów VRP. Dane wejściowe dla cVRP składają się z n lokalizacji (Depot oraz zbiór $n - 1$ klientów), symetrycznej macierzy odległości D o wymiarach $n \times n$ określającej koszt podróży między każdą parą lokalizacji, zapotrzebowania q_i na pewien zasób przez każdego klienta i oraz maksymalnej pojemności pojazdu Q .

Rozwiązanie dopuszczalne dla cVRP składa się ze zbioru tras, które rozpoczynają się i kończą w depocie, przy czym każdy klient jest odwiedzany dokładnie raz, a całkowite zapotrzebowanie klientów przypisanych do danej trasy nie przekracza pojemności pojazdu Q . Rozwiązanie optymalne minimalizuje całkowitą sumę długości wszystkich tras. W ramach niniejszego projektu nie nakłada się ograniczeń na liczbę tras w rozwiązaniu.

Problem cVRP należy do klasy problemów NP-trudnych, co oznacza, że nie istnieje znany algorytm wielomianowy gwarantujący znalezienie rozwiązania optymalnego. Z tego powodu dla instancji o więk-

szych rozmiarach stosuje się metaheurystyki, które w akceptowalnym czasie są w stanie wygenerować rozwiązania o dobrej jakości, choć niekoniecznie optymalne.

3 Zastosowane algorytmy

W ramach zrealizowanego doświadczenia przetestowano cztery podejścia, które różnią się poziomem zaawansowania oraz, co zostanie wykazane w dalszej części tej pracy, jakością generowanych przez nie rozwiązań. Poniższy rozdział w skrócie opisuje wspomniane podejścia.

3.1 Algorytm losowy

Algorytm losowy stanowi najprostsze możliwe podejście do rozwiązania problemu cVRP i służy jako punkt odniesienia dla bardziej zaawansowanych metod. W każdej iteracji algorytm losowo permutuje listę klientów, a następnie konstruuje trasy poprzez sekwencyjne przydzielanie klientów do pojazdów, respektując ograniczenie pojemności. Gdy dodanie kolejnego klienta przekroczyłoby pojemność aktualnego pojazdu, rozpoczynana jest nowa trasa.

Algorytm nie wykorzystuje żadnej informacji o strukturze problemu, takiej jak odległości między klientami czy ich zapotrzebowanie. Z tego powodu generowane rozwiązania są zazwyczaj daleko od optymalnych. Niemniej algorytm losowy jest użyteczny jako punkt odniesienia do oceny skuteczności innych metod oraz do weryfikacji poprawności implementacji ograniczeń problemu.

3.2 Algorytm zachłanny

Algorytm zachłanny konstruuje rozwiązanie iteracyjnie, w każdym kroku podejmując decyzję, która wydaje się lokalnie optymalna. W kontekście cVRP algorytm rozpoczyna trasę od magazynu i w każdym kroku wybiera najbliższego, jeszcze nieodwiedzanego klienta, którego zapotrzebowanie nie przekracza dostępnej pojemności pojazdu. Gdy nie można dodać żadnego klienta do aktualnej trasy bez naruszenia ograniczenia pojemności, pojazd wraca do magazynu i rozpoczyna się nowa trasa.

W przeprowadzonym doświadczeniu wprowadzono modyfikację algorytmu zachłannego poprzez wymuszenie na nim pierwszego odwiedzanego miasta. Algorytm zawsze rozpoczyna z magazynu, lecz w przeprowadzanych badaniach przy każdym uruchomieniu udaje się do innego, wymuszonego na nim, pierwszego klienta. Modyfikacja ma na celu wprowadzenie większej różnorodności wśród rozwiązań generowanych przez algorytm oraz zwiększa szanse na ominięcie pierwszego lokalnego minimum jakie może być spowodowane wyborem pierwszego najbliższego miasta. Efektywnie, otrzymamy n wyników gdzie n to wielkość instancji w rozumieniu liczby klientów. Takie działanie algorytmu zachłannego spowodowane jest tym, że jako jedyny spośród analizowanych jest **deterministyczny**.

3.3 Algorytm Symulowanego Wyżarzania

Symulowane wyżarzanie (*ang. Simulated Annealing*, SA) jest metaheurystyką inspirowaną procesem wyżarzania metali, w którym materiał jest powoli chłodzony, aby osiągnąć stan o minimalnej energii. Algorytm rozpoczyna od rozwiązania początkowego (np. wygenerowanego algorytmem zachłannym) i iteracyjnie generuje sąsiednie rozwiązania poprzez zastosowanie lokalnych modyfikacji, takich jak zamiana kolejności klientów lub przeniesienie klienta między trasami.

Kluczową cechą SA jest mechanizm akceptacji gorszych rozwiązań z pewnym prawdopodobieństwem, które zależy od parametru temperatury, a które maleje w trakcie działania algorytmu. Na początku, przy wysokiej temperaturze, algorytm akceptuje większość ruchów, co umożliwia eksplorację przestrzeni rozwiązań. Wraz ze spadkiem temperatury algorytm staje się bardziej zachowawczy, koncentrując się na eksploatacji obiecujących regionów. Taki mechanizm pozwala uniknąć utknięcia w lokalnych optimum i zwiększa szansę znalezienia rozwiązania bliskiego globalnemu optimum.

3.4 Metaheurystyka genetyczna

Algorytm genetyczny (EA, *ang. Evolutionary Algorithm*) to metaheurystyka inspirowana procesem ewolucji biologicznej, operująca na populacji rozwiązań. Każde rozwiązanie (osobnik) jest kodowane jako

chromosom, reprezentujący kolejność odwiedzania klientów. Algorytm rozpoczyna od losowo wygenerowanej populacji początkowej, a następnie w kolejnych pokoleniach stosuje operatory genetyczne: selekcję, krzyżowanie i mutację.

Selekcja faworyzuje osobniki o lepszej funkcji przystosowania (niższym koszcie tras), które mają większą szansę na reprodukcję. Krzyżowanie (*crossover*) łączy cechy dwóch rodziców, tworząc potomstwo, które dziedziczy fragmenty tras od obojga. Mutacja wprowadza losowe zmiany, takie jak zamiana pozycji klientów, zapewniając różnorodność genetyczną. Po wygenerowaniu potomstwa stosuje się mechanizm sukcesji, który określa, które osobniki przechodzą do następnego pokolenia. Dzięki równoważeniu eksploracji (poprzez krzyżowanie i mutację) oraz eksploatacji (poprzez selekcję) algorytm genetyczny jest w stanie efektywnie przeszukiwać złożoną przestrzeń rozwiązań problemu cVRP.

4 Implementacja

4.1 Język Go

Do realizacji przedstawianego doświadczenia wykorzystano język programowania **Go** (znany także jako *golang*). Zdecydowano się na niego ze względu na to, że jest popularną alternatywą dla języków niskopoziomowych, takich jak C lub C++. Do jego zalet należy nowoczesna składnia wyglądem zbliżona do Python'a, obecność *garbage collector'a* oraz minimalna konfiguracja wymagana do uruchomienia projektu - w zasadzie wystarcza jeden plik `go.mod`.

Go posiada również bardzo przydatny i wydajny sposób zrównoleglania procesów – *goroutines*. Zostały one wykorzystane do równoczesnego uruchamiania różnych algorytmów dla jednej instancji oraz równoległego uruchamiania tego samego algorytmu w celu wyznaczania średnich wyników. Dzięki temu możliwe było znaczące przyspieszenie procesu przeprowadzania eksperymentów.

Minusem Go z perspektywy programisty, który wcześniej w nim nie pisał, jest brak typowego podejścia obiektowego. W Go nie ma klas ani dziedziczenia. Polimorfizm jest osiągany przez interfejsy, które jednak nie wymuszają sztywnej implementacji metod w danej strukturze. Założeniem twórców było usunięcie z języka wszystkich elementów, które nie są intuicyjne i często prowadzą do problemów. Z tego powodu krzywa uczenia się języka jest początkowo stroma – należy nauczyć się, jak zastąpić typowe konstrukcje, takie jak klasy enumeracyjne, oraz zmienić obiektowe myślenie o projektowaniu architektury aplikacji na bardziej funkcyjne.

Poniżej przedstawiono przykładową strukturę reprezentującą instancję problemu cVRP oraz sposób definiowania metod w języku Go:

```
1 type Instance struct {
2     Name          string
3     NodesMatrix   []Node
4     DistanceMatrix [][]float32
5     TruckMaxLoad  int
6 }
7
8 func (i Instance) GetNodesDistance(n1 Node, n2 Node) float32 {
9     return i.DistanceMatrix[n1.ID][n2.ID]
10 }
11
12 // GetDepotID returns the depot ID, which is always 0;
13 // other nodes are numbered 1..n.
14 func (i Instance) GetDepotID() int {
15     return 0
16 }
17
18 func (i Instance) IsCapacityExceeded(route []Node) bool {
19     totalDemand := 0
20     for _, node := range route {
21         totalDemand += node.Demand
22     }
23     return totalDemand > i.TruckMaxLoad
24 }
```

Struktura `Instance` nie zawiera zdefiniowanych metod w swoim ciele - metody są *podpinane* do niej z zewnątrz poprzez dodanie (*receiver*) w definicji funkcji. Technicznie jest to definicja funkcji, która przyjmuje wskaźnik lub wartość instancji struktury jako pierwszy (ukryty) parametr. Jest to jedna z wielu różnic składniowo-implementacyjnych względem języków takich jak Java, C++ czy TypeScript, gdzie metody są jawnie częścią definicji klasy.

Taki mechanizm pozwala na bardziej elastyczne projektowanie API - metody mogą być dodawane w różnych plikach i pakietach, co sprzyja modularności kodu. Jednocześnie wymaga to od programisty innego sposobu myślenia o organizacji kodu, bardziej zbliżonego do paradygmatu funkcyjnego niż obiektowego.

4.2 Implementacje metaheurystyk

W tej sekcji opisane będą decyzje implementacyjne metaheurystyk - genetycznej oraz symulowanego wyżarzania - wraz z uwzględnieniem konfigurowalnych dla nich parametrów. Ponadto porusza ona kwestię sposobu generowania rozwiązań zgodnych z założeniami problemu cVRP.

4.2.1 Realizacja generowania rozwiązań

Problem cVRP został uproszczony do problemu komiwojażera (*ang. Traveling Salesman Problem - TSP*). Metaheurystyka genetyczna oraz metaheurystyka symulowanego wyżarzania generują listę miast do odwiedzenia, a funkcja oceny rozwiązań przekształca daną listę w sposób zachłanny w legalne rozwiązanie problemu cVRP. Proces przerabiania opiera się na następujących założeniach:

- Algorytm zawsze zaczyna z magazynu
- algorytm odwiedza kolejne sklepy z listy, do czasu gdy zapotrzebowanie następnego sklepu jest większe niż zapas w samochodzie. wtedy udaje się do magazynu
- po uzupełnieniu zapasów algorytm kontynuuje odwiedzanie sklepów w kolejności zgodnej z listą
- po odwiedzeniu ostatniego sklepu algorytm wraca do magazynu

4.2.2 Metaheurystyka Genetyczna

W implementacji zdecydowano się na realizację **mutacji** poprzez zamianę dwóch losowo wybranych miast. Operacja krzyżowania dwóch rozwiązań została zrealizowana poprzez wykorzystanie operatora `Order Crossover (OX)`. Wybór osobników do krzyżowania następuje przy użyciu operatora selekcji, zrealizowanego metodą turnieju.

Konfiguracja następuje z wykorzystaniem poniższego zestawu parametrów:

- `PopulationSize` - wielkość populacji w każdej generacji
- `MaxGenerations` - ilość generacji
- `MutationRate` - prawdopodobieństwo wystąpienia mutacji przy tworzeniu nowego osobnika
- `CrossoverRate` - prawdopodobieństwo wystąpienia krzyżowania przy tworzeniu nowego osobnika
- `TournamentSize` - wielkość turnieju przy wyborze partnera do krzyżowania
- `ElitismCount` - elityzm, definiuje ile osobników z najlepszym wynikiem funkcji dopasowania zostanie zachowanych w kolejnej generacji

4.2.3 Metaheurystyka symulowanego wyżarzania

W implementacji algorytmu symulowanego wyżarzania wykorzystano **perturbację** polegającą na zamianie dwóch losowo wybranych klientów w trasie. Rozwiązanie początkowe generowane jest za pomocą algorytmu zachłannego, startując od losowo wybranego węzła. Akceptacja gorszych rozwiązań odbywa się zgodnie z kryterium Metropolis - nowe rozwiązanie jest akceptowane z prawdopodobieństwem $e^{-\Delta/T}$, gdzie Δ to wzrost kosztu, a T to aktualna temperatura.

Proces wyżarzania przebiega iteracyjnie - w każdej iteracji temperatura jest mnożona przez współczynnik chłodzenia α , co zmniejsza prawdopodobieństwo akceptacji gorszych rozwiązań. Dla każdej wartości temperatury wykonywana jest stała liczba perturbacji (pętla wewnętrzna), co pozwala na dokładniejsze

przeszukanie przestrzeni rozwiązań przy danym poziomie temperatury. Algorytm kończy działanie, gdy temperatura spadnie poniżej zadanego minimum.

Konfiguracja następuje z wykorzystaniem poniższego zestawu parametrów:

- **InitialTemperature** - temperatura początkowa, determinująca początkową skłonność do akceptacji gorszych rozwiązań
- **CoolingRate** - współczynnik chłodzenia $\alpha \in (0, 1)$, określający tempo spadku temperatury
- **MinimumTemperature** - temperatura minimalna, przy której algorytm kończy działanie
- **InnerLoop** - liczba perturbacji wykonywanych dla każdej wartości temperatury (iteracje wewnętrzne)

5 Doświadczenie

W ramach przeprowadzania doświadczenia wykonano następujące kroki:

- Implementacja algorytmów w języku Go
- Manualne dostrajanie parametrów metaheurystyki genetycznej oraz symulowanego wyżarzania
- przeprowadzenie eksperymentu generalnego - przy określonych parametrach uruchomienie dla każdego algorytmu kilku, z góry określonych instancji problemu
- przeprowadzenie analizy poprawy rozwiązania z kolejnymi iteracjami algorytmów

Poniższe podrozdziały szerzej opisują założenia, proces realizacji oraz otrzymane wyniki doświadczenia. Algorytmy były testowane na poniższym zestawie instancji:

- A-n32-k5
- A-n37-k6
- A-n39-k5
- A-n45-k6
- A-n48-k7
- A-n54-k7
- A-n60-k9

Pliki definiujące instancje problemu pozyskane zostały z <http://vrp.galagos.inf.puc-rio.br/index.php/en/>

5.1 Dostrajanie parametrów metaheurystyk

Dostrajanie przeprowadzono na najmniejszej instancji spośród wymaganych - A-n32-k5. Celem było uzyskanie akceptowalnego, krótkiego czasu wykonania algorytmu, rzędu około 1 minuty. Drugim aspektem, na który zwracano uwagę podczas dostrajania, było zbliżenie się do optymalnego rozwiązania zdefiniowanego w pliku z problemem. W ten sposób wyznaczone zostały poniższe zestawy:

Parametry dla metaheurystyki genetycznej

- **PopulationSize** = 400
- **MaxGenerations** = 100000
- **MutationRate** = 0.01
- **CrossoverRate** = 0.95
- **TournamentSize** = 10
- **ElitismCount** = 10

Parametry dla metaheurystyki symulowanego wyżarzania

- InitialTemperature = 2000
- CoolingRate = 0.9999
- MinimumTemperature = 0.001
- InnerLoop = 2000

5.2 Wyniki eksperymentów

5.2.1 Eksperyment generalny

Czas wykonania badania w zależności od wielkości instancji Ze względu na skuteczne zrównoleglenie wykonywanych operacji, czas wykonania pojedynczego badania zawierał się w przedziale od 43 sekund do 1 minuty i 15s. Zysk, jaki otrzymujemy względem czasu bez zrównoleglenia, można oszacować na około 18-20 razy czasu, jaki byłby potrzebny do wykonania sekwencyjnie 10 pomiarów dla metaheurystyki genetycznej, a następnie 10 pomiarów dla metaheurystyki symulowanego wyżarzania. Poniższa tabela prezentuje czas potrzebny na realizację zestawów poszczególnych badań w zależności od wielkości instancji problemu.

Tabela 1: Czasy wykonania testowanych algorytmów dla różnych instancji problemu VRP

Problem	Greedy [ms]	Random [ms]	Genetyczny [s]	SA [s]
a-n32-k5	4.32	28.45	36.41	43.10
a-n37-k6	3.68	55.29	48.64	59.56
a-n39-k5	13.33	52.11	43.69	53.79
a-n45-k6	7.09	50.59	45.15	56.46
a-n48-k7	8.76	35.86	40.91	54.35
a-n54-k7	8.68	45.20	50.50	65.37
a-n60-k9	10.75	39.22	71.64	73.83

Uwaga: Algorytm **Greedy** został wykonany n -krotnie, gdzie n oznacza liczbę miast (wartość ta znajduje się w nazwie pliku z danymi). Algorytm **Random** został uruchomiony **10 000 razy**. Algorytmy **genetyczny** oraz **symulowanego (SA)** wykonano po **10 razy każdy**, przy czym obliczenia były **zrównoleglone**.

5.3 Średnie wyniki dla poszczególnych instancji

Tabela 2: Porównanie wyników algorytmów

Problem	Opt. result	Random [10k]				Greedy [n]				Evolutionary [10x]				SA [10x]			
		best	worst	avg	std	best	worst	avg	std	best	worst	avg	std	best	worst	avg	std
n32-k5	784	1569	2585	2119	140	906	1143	1011	50	800	861	819	17	800	820	807	6
n37-k6	949	1698	2658	2220	129	1063	1239	1160	47	958	997	973	12	951	976	958	8
n39-k5	822	1582	2601	2184	130	924	1308	1088	97	831	942	873	30	832	860	838	8
n45-k6	944	2252	3322	2819	153	1122	1340	1227	63	959	1067	997	33	957	1002	971	13
n48-k7	1073	2319	3554	2941	168	1260	1423	1345	42	1112	1217	1149	28	1074	1140	1097	17
n54-k7	1167	2626	3896	3289	171	1385	1694	1525	92	1191	1240	1212	17	1176	1199	1187	7
n60-k9	1354	2970	4376	3701	179	1548	1791	1648	46	1366	1464	1417	25	1363	1401	1383	12

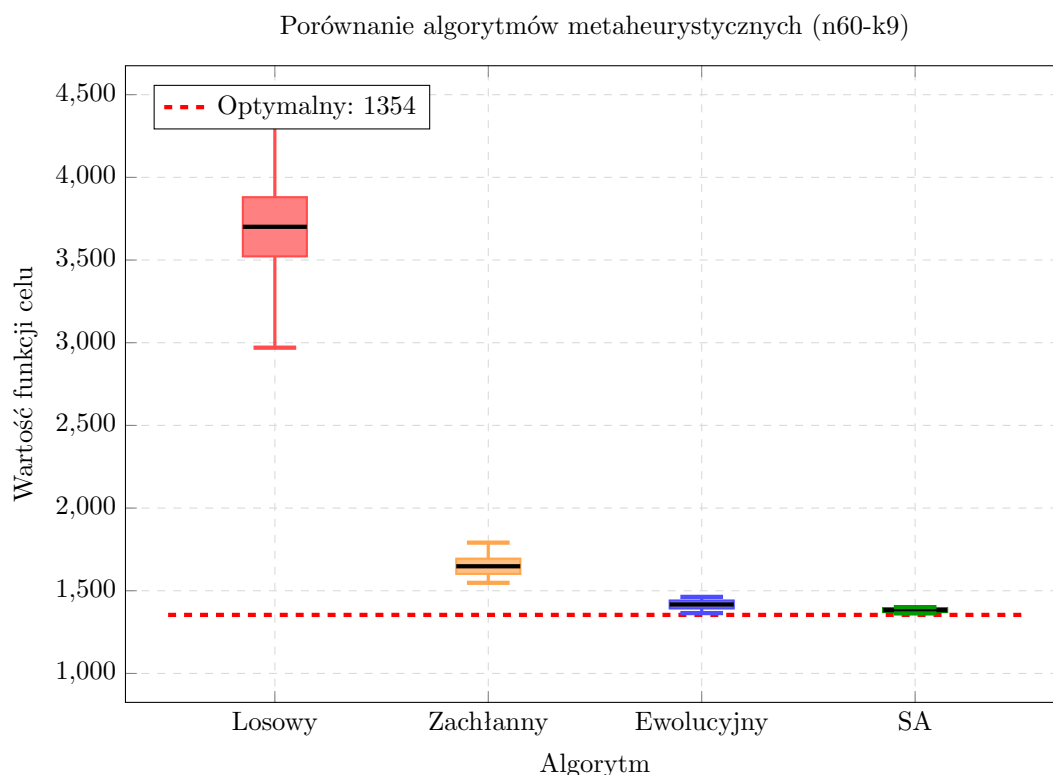
Tabela 3: Porównanie wyników algorytmów - dane przekształcone

Problem	Opt.	Random [10k]	Greedy [n]	Evolutionary [10x]	SA [10x]
n32-k5	784	2119 ± 140	906	819 ± 17	807 ± 6
n37-k6	949	2220 ± 129	1063	973 ± 12	958 ± 8
n39-k5	822	2184 ± 130	924	873 ± 30	838 ± 8
n45-k6	944	2819 ± 153	1122	997 ± 33	971 ± 13
n48-k7	1073	2941 ± 168	1260	1149 ± 28	1097 ± 17
n54-k7	1167	3289 ± 171	1385	1212 ± 17	1187 ± 7
n60-k9	1354	3701 ± 179	1548	1417 ± 25	1383 ± 12

Wyniki przedstawione w tabeli są zaokrąglone do wartości całkowitych. Pogrubione zostały najlepsze rozwiązania znalezione dla każdego problemu. Na podstawie zebranych informacji można wyciągnąć następujące wnioski:

- Metaheurystyka ewolucyjna znalazła najlepsze spośród zastosowanych podejść tylko w dwóch z siedmiu przypadków
- Metaheurystyka Symulowanego Wyżarzania (SA) znalazła najlepsze rozwiązanie w sześciu z siedmiu przypadków
- Algorytm losowy i zachłanny nie znalazły ani razu najlepszego z rozwiązań
- Rozwiązania znajdowane przez metaheurystykę SA mają niższe odchylenie standardowe niż rozwiązania otrzymywane podejściem ewolucyjnym. Ponadto, jak zostało wskazane wcześniej, w większości przypadków były one najbliższe rozwiązaniu optymalnemu.

W oparciu o powyższe obserwacje, spośród zaimplementowanych podejść, metaheurystyka symulowanego wyżarzania sprawdza się najlepiej. Warto zauważyć jednak, że podane testy zostały przeprowadzone tylko dla jednego zestawu parametrów dostrajania algorytmu, a manipulując tymi parametrami, można prawdopodobnie wpłynąć na poprawę wyników.

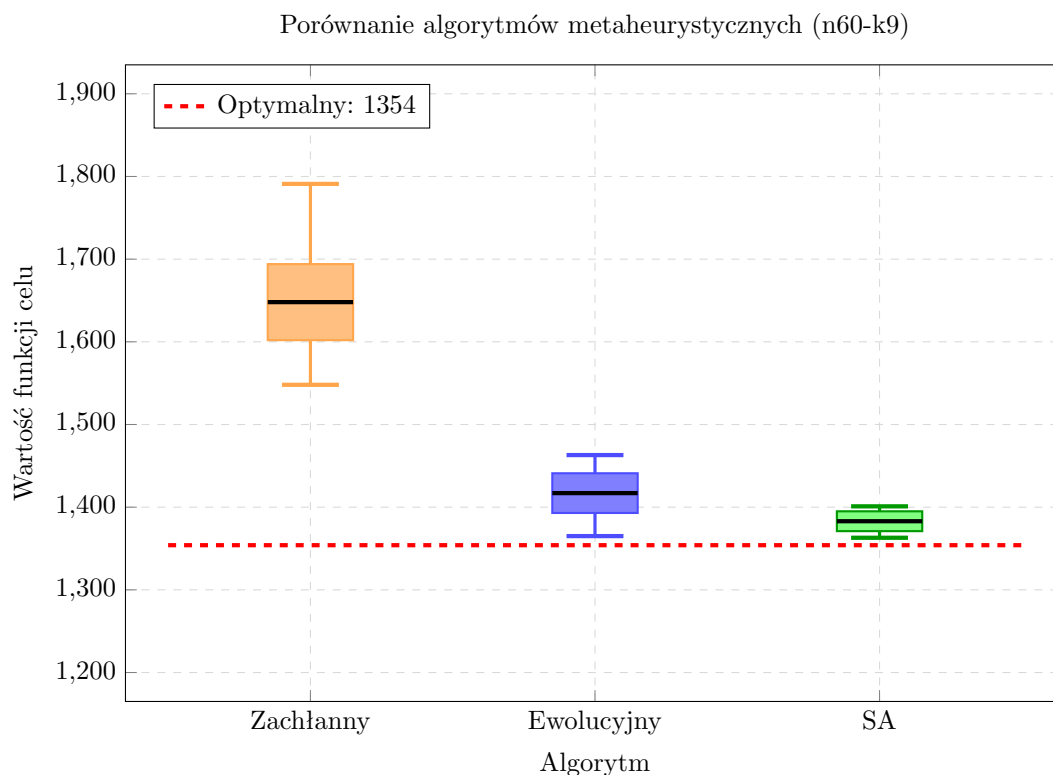


Rysunek 1: Wykres pudełkowy porównujący wyniki algorytmów dla instancji n60-k9

Na wykresach 2 oraz 1 przedstawiono wyniki pomiarów dla największej z testowanych prób. Wykres 2 przedstawia wyniki dla czterech podejść, podczas gdy wykres 1, w celu poprawy czytelności, pomija wyniki dla algorytmu losowego.

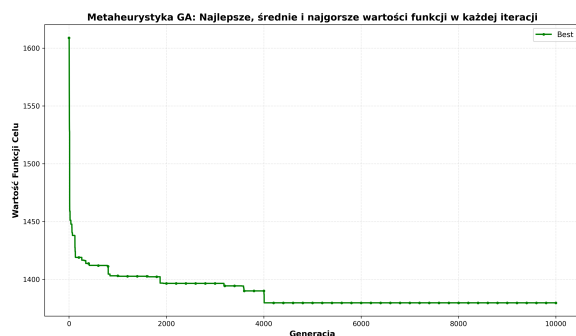
Poniższe wykresy prezentują jak kształtowały się wartości funkcji oceny rozwiązania dla rozwiązań generowanych przez metaheurystyki: Genetyczną i Symulowane Wyżarzanie (SA).

Rysunki 3 i 4 przedstawiają przebieg wartości najlepszego rozwiązania w kolejnych iteracjach algorytmów. Na wykresach tych można zaobserwować charakterystyczne zachowanie obu metaheurystyk. Algorytm genetyczny (rysunek 3) wykazuje stopniową, ale konsekwentną poprawę jakości rozwiązania, co jest typowe dla metod ewolucyjnych, w których selekcja i operatory genetyczne sukcesywnie prowadzą populację w kierunku lepszych obszarów przestrzeni rozwiązań. Symulowane wyżarzanie (rysunek 4) przedstawia drastyczny spadek wartości funkcji dopasowania, po którym algorytm nie znajduje już lepszego rozwiązania.

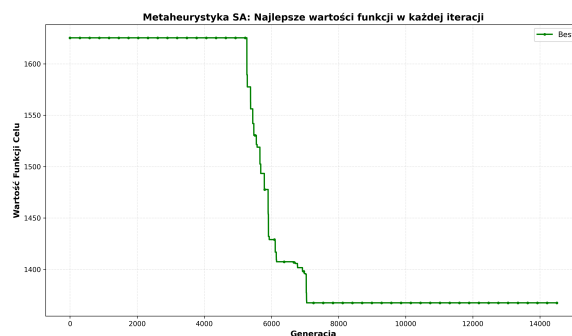


Rysunek 2: Wykres pudełkowy porównujący wyniki algorytmów dla instancji n60-k9 z pominięciem algorytmu losowego

Początkowo, również przez prawie połowę wykonywanych iteracji, nie znajduje lepszego rozwiązania niż rozwiązanie początkowe. Jest to spowodowane jakością pierwszego rozwiązania, które zostało wyznaczone metodą zachłanną.



Rysunek 3: Przebieg wartości najlepszego rozwiązania dla algorytmu genetycznego

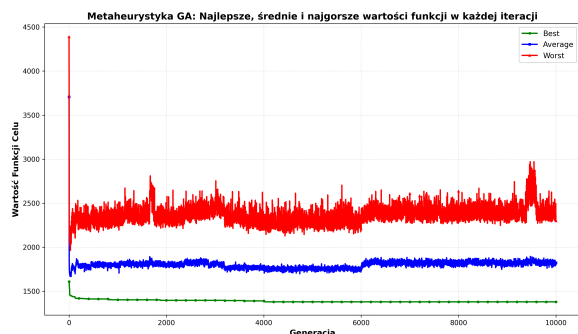


Rysunek 4: Przebieg wartości najlepszego rozwiązania dla symulowanego wyżarzania

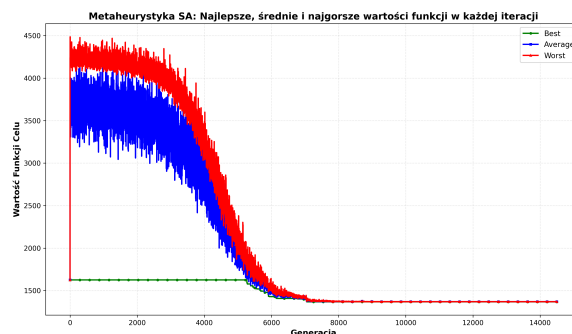
Rysunki 5 i 6 prezentują pełniejszy obraz procesu optymalizacji, uwzględniając nie tylko najlepsze rozwiązania, ale również wartości średnie i najgorsze w danej iteracji. Analiza tych wykresów pozwala na ocenę stabilności i różnorodności populacji rozwiązań. W przypadku algorytmu genetycznego (rysunek 5) obserwuje się brak zbieżności krzywych, co świadczy o błędnym doborze parametrów dla algorytmu. Dla symulowanego wyżarzania (rysunek 6) wykres pokazuje większą zbieżność. Początkowe rozwiązanie, wyznaczone zachłannie, przez długi czas nie zostaje pobite. Rozwiązania średnie i najgorsze z biegiem czasu dążą do najlepszego, a w miejscu ich spotkania widać spadek wartości krzywej rozwiązania najlepszego.

6 Podsumowanie i wnioski

W oparciu o przedstawione wyniki badań stwierdza się, że:



Rysunek 5: Przebieg wartości najlepszego, średniego i najgorszego rozwiązania dla algorytmu genetycznego



Rysunek 6: Przebieg wartości najlepszego, średniego i najgorszego rozwiązania dla symulowanego wyżarzania

- Metaheurystyka symulowanego wyżarzania uzyskała najlepsze wyniki spośród testowanych algorytmów, a poprawność jej działania jest potwierdzona na wykresach.
- Metaheurystyka genetyczna uzyskiwała akceptowalne wyniki, znacząco lepsze niż wyniki algorytmu zachłannego, lecz zazwyczaj gorsze niż metaheurystyka symulowanego wyżarzania, z większą niż ona zmiennością. Przedstawione wykresy 5 oraz 3 uwiadcniają problemy algorytmu, które wynikają prawdopodobnie zarówno z jego implementacji, jak i doboru parametrów.
- Algorytm zachłanny generował wyniki znacząco słabsze od metaheurystyk, lecz również znacząco lepsze niż algorytm losowy, którego zastosowanie tutaj ma miejsce wyłącznie ze względów doświadczalnych. Nie nadaje się do rozwiązywania problemów tego typu i nie powinien być do tego stosowany.
- wykorzystanie języka **Go (golang)** ułatwiło implementację zrównoleżenia operacji, a wydajność oferowana przez napisany program pozwala na sprawne wykonanie testów.

Ponadto, po wykonaniu wstępnych badań, przeprowadzono dostrajanie parametrów, które wykazało:

- algorytm genetyczny radzi osiągał lepsze wyniki przy niższym współczynniku mutacji
- obydwa algorytmy osiągały lepsze wyniki jeśli wśród ich początkowych rozwiązań znajdowało się rozwiązanie zachłanne
- pomimo dostrajania parametrów nie udało się poprawić przebiegu funkcji celu algorytmu genetycznego

Kod zrealizowany w ramach tego projektu jest dostępny w repozytorium GitHub pod linkiem <https://github.com/skurzyp/cvrp-metaheuristics>.

A Suplement: Analiza parametrów algorytmu ewolucyjnego

A.1 Wpływ rozmiaru turnieju na działanie EA

Rozmiar turnieju (Tour) kontroluje presję selekcyjną w algorytmie ewolucyjnym. Gdy Tour=1, selekcja staje się całkowicie losowa, co prowadzi do zachowania podobnego do przeszukiwania losowego. Gdy Tour=pop_size, wybieramy zawsze najlepszego osobnika z całej populacji, co powoduje bardzo wysoką presję selekcyjną i ryzyko utknięcia w lokalnym optimum z powodu przedwczesnej utraty różnorodności genetycznej.

A.2 Różnica między EA z Tour=1 a Tour_i

EA z Tour=1 nie stosuje faktycznej selekcji – każdy osobnik ma równe szanse na reprodukcję, niezależnie od jakości rozwiązania, co przekształca algorytm w generator losowych wariantów z krzyżowaniem i mutacją. EA z Tour_i wprowadza presję selekcyjną, gdzie lepsze osobniki mają większe szanse na reprodukcję, co kierunkuje poszukiwania w stronę obiecujących obszarów przestrzeni rozwiązań.

A.3 Poprawność działania ruletki dla problemu cVRP

Ruletka nie działa poprawnie dla problemu cVRP, ponieważ jest to problem minimalizacji, w którym mniejsze wartości funkcji celu są lepsze. Standardowa ruletka zakłada maksymalizację – przydziela większe prawdopodobieństwo selekcji osobnikom o wyższych wartościach funkcji oceny rozwiązania, co w cVRP faworyzowałoby najgorsze rozwiązania.

A.4 Czy mutacji może być za mało lub za dużo?

Tak, zarówno zbyt niska, jak i zbyt wysoka częstość mutacji negatywnie wpływa na działanie EA. Zbyt mała mutacja ($P_m \rightarrow 0$) ogranicza zdolność algorytmu do eksploracji nowych obszarów przestrzeni rozwiązań. Zbyt wysoka mutacja ($P_m \rightarrow 1$) przekształca algorytm w przeszukiwanie losowe.

A.5 Rola operatora mutacji

Mutacja umożliwia algorytmowi ucieczkę z lokalnych optima przez losowe modyfikacje osobników, utrzymując różnorodność genetyczną populacji i zapewniając, że każdy punkt przestrzeni rozwiązań może być teoretycznie osiągnięty.

A.6 Czy krzyżowania może być za mało lub za dużo?

Tak, niewłaściwy poziom krzyżowania osłabia efektywność EA. Zbyt niskie P_x ogranicza wymianę informacji genetycznej między osobnikami, zmniejszając eksploatację dobrych schematów i spowalniając konwergencję do dobrych rozwiązań. Zbyt wysokie P_x (bliskie 1.0) może destrukcyjnie wpływać na najlepsze osobniki w populacji, niszcząc wypracowane dobre struktury rozwiązań częściej niż pozwalając im się utrwalić przez elityzm i selekcję.

A.7 Rola operatora krzyżowania

Operator krzyżowania pełni rolę mechanizmu eksploatacji poprzez rekombinację materiału genetycznego z różnych osobników w celu tworzenia potomstwa potencjalnie dziedziczącego korzystne cechy obojga rodziców.

A.8 Konsekwencje wyłączenia krzyżowania i/lub mutacji

Wyłączenie obu operatorów ($P_x=0$, $P_m=0$) redukuje EA do wielokrotnej selekcji z początkowej populacji losowej, co eliminuje możliwość poprawy rozwiązań i prowadzi do stagnacji na poziomie najlepszego osobnika z inicjalizacji. Wyłączenie tylko krzyżowania ($P_x=0$, $P_m \neq 0$) przekształca algorytm w przeszukiwanie losowe z selekcją, tracąc zdolność do budowania rozwiązań poprzez rekombinację. Wyłączenie tylko mutacji ($P_m=0$, $P_x \neq 0$) drastycznie ogranicza różnorodność genetyczną w materiale, powodując brak możliwości uzyskania wszystkich możliwych rozwiązań oraz zwiększając szansę utknięcia w minimum lokalnym.

A.9 Potrzeba selekcji elitarniej

Bez elityzmu istnieje ryzyko utraty najlepszego znalezionej rozwiązania w wyniku operacji genetycznych lub selekcji. Elityzm stabilizuje algorytm i zapewnia zabezpieczenie przed regresją jakości.

A.10 Wpływ rozmiaru populacji na efektywność EA

Rozmiar populacji i liczba generacji determinują balans między jakością rozwiązań a kosztem obliczeniowym algorytmu. Mała populacja prowadzi do szybkiej utraty różnorodności genetycznej oraz utknięcia w lokalnych optimum, ale wymaga mniej obliczeń na generację. Duża populacja utrzymuje większą różnorodność, lepiej eksploruje przestrzeń rozwiązań i ma większą szansę znalezienia globalnego optimum, ale wymaga znacznie więcej obliczeń i pamięci.

Bibliografia

- [1] *Vehicle Routing Problem Repository*, <https://vrp.galgos.inf.puc-rio.br/index.php/en/>, [dostęp: 15.10.2025].
- [2] *DIMACS Implementation Challenge: Vehicle Routing Problem*, Rutgers University, <http://dimacs.rutgers.edu/programs/challenge/vrp/cvrp/>, [dostęp: 15.10.2025].
- [3] *VRP-REP: Vehicle Routing Problem Repository - Capacitated VRP*, <http://www.vrp-rep.org/variants/item/cvrp.html>, [dostęp: 15.10.2025].